PHIẾU THỰC HÀNH — 09/10/2026

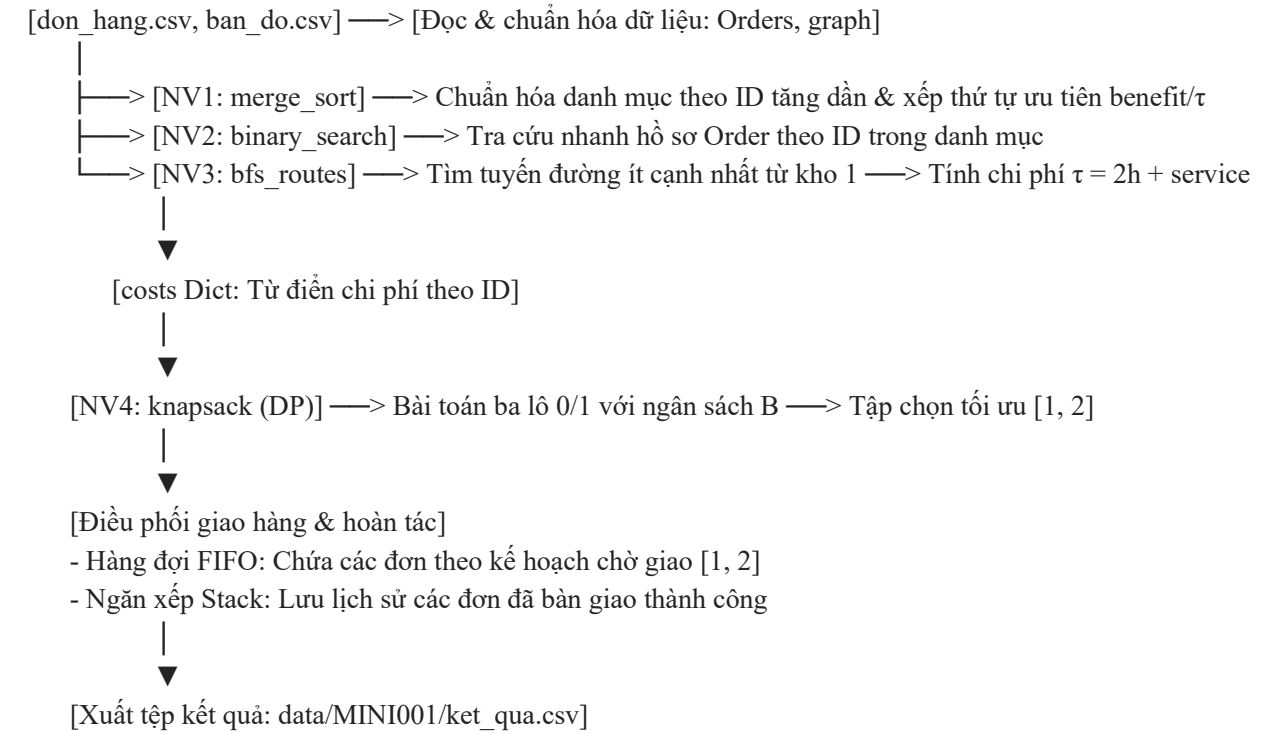
Họ tên: Đỗ Hồng Dương **MSSV:** N24DCDT018 **Lớp:** D24CQDT01-N

GitHub: https://github.com/ResitaRias/thuc_hanh_ctdl_buoi05 **Commit SHA:** 3847dab0c0561dcdd9141f979e40adc4acb5249c

1. Mô tả một luồng ứng dụng

Ghi mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng.

• Mục tiêu: Bài toán xây dựng hệ thống điều phối và thực thi các đơn giao hàng trong khuôn viên trường từ kho trung tâm (đỉnh 1). Yêu cầu cốt lõi là chọn ra một tập hợp các đơn hàng thỏa mãn ràng buộc tổng thời gian phục vụ không vượt quá ngân sách B (ở bộ mini B = 8), đồng thời đạt tổng mức độ lợi ích thu được là cực đại.
• Đầu vào: Hai tệp dữ liệu CSV gồm don_hang.csv (danh sách đơn hàng chứa ID, điểm giao destination, thời gian phục vụ service, giá trị lợi ích benefit) và ban_do.csv (mạng lưới giao thông nội bộ mô tả bằng các cạnh vô hướng nối giữa các đỉnh); kèm theo tham số ngân sách thời gian B.
• Đầu ra: Tập các đơn hàng tối ưu được phê duyệt thực hiện, đường đi ngắn nhất (ít cạnh nhất) và chi phí thời gian khứ hồi τ cho từng đơn; nhật ký mô phỏng các biến đổi trạng thái của hàng đợi giao hàng (FIFO) và ngăn xếp hoàn tác (Stack); tệp dữ liệu kết quả ket_qua.csv ghi nhận đầy đủ 9 cột thông số.
• Sơ đồ truyền dữ liệu giữa các khối chức năng:



2. Bảng truy vết đường và thời gian

ID	Đường lượt đi	Số cạnh h	2h + phục vụ (τ)
1	1 -> 2	1	2 × 1 + 1 = 3
2	1 -> 2 -> 4	2	2 × 2 + 1 = 5

ID	Đường lượt đi	Số cạnh h	2h + phục vụ (τ)
3	1 -> 2 -> 4 -> 6	3	2 × 3 + 1 = 7

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

Ý tưởng/điều kiện áp dụng: Áp dụng nguyên lý chia để trị (Divide and Conquer). Nếu dãy có độ dài ≤ 1 thì trả về collect(a). Ngược lại, xác định vị trí trung vị mid = div(length(a), 2) để chia mảng thành hai nửa left và right, gọi đệ quy sắp xếp từng nửa rồi dùng hai con trỏ để trộn (merge) các phần tử theo hàm so sánh before. Điều kiện: Quan hệ so sánh before nhất quán; thuật toán bảo đảm tính ổn định (stable) và không làm biến đổi mảng gốc ban đầu.

Lời gọi và kết quả kiểm thử: Chạy lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=sort. Kết quả: ĐẠT toàn bộ các bài test (kiểm tra mảng rỗng, mảng 1 phần tử, mảng số nguyên đảo ngược, sắp danh mục Order theo tiêu chí (x, y) -> x.id < y.id). Dãy kết quả chuẩn xác, bảo toàn số lượng phần tử và độc lập với mảng đầu vào.

Chi phí thời gian/bộ nhớ: Thời gian: O(n log n) trong mọi trường hợp (tốt nhất, trung bình và xấu nhất) do cây đệ quy có độ sâu log n và chi phí trộn mỗi tầng là O(n). Bộ nhớ phụ: O(n) để cấp phát các mảng con trong quá trình phân chia và kết quả trộn.

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

Ý tưởng/điều kiện áp dụng: Thu hẹp không gian tìm kiếm nhị phân với hai con trỏ lo = 1 và hi = length(a). Lặp khi lo <= hi: tính mid = lo + div(hi - lo, 2). Nếu a[mid].id == id trả về a[mid]; nếu a[mid].id < id dời lo = mid + 1; ngược lại dời hi = mid - 1. Kết thúc vòng lặp không thấy trả về nothing. Điều kiện: Dãy a bắt buộc phải được sắp xếp tăng dần theo trường khóa id duy nhất và danh mục không đổi trong quá trình tìm kiếm.

Lời gọi và kết quả kiểm thử: Chạy lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=search. Kết quả: ĐẠT 100% các phép thử (tìm thấy chính xác các đơn hàng ở đầu, giữa, cuối danh mục trả về đúng kiểu Order; tra cứu ID không tồn tại như ID 99 hoặc trên danh mục rỗng trả về nothing an toàn).

Chi phí thời gian/bộ nhớ: Thời gian: O(log n) do không gian tìm kiếm giảm một nửa sau mỗi phép so sánh. Bộ nhớ phụ: O(1) do chỉ sử dụng vài biến chỉ số cục bộ trên ngăn xếp.

NV3 · Hoàn thiện bfs_routes(graph, source=1)

Ý tưởng/điều kiện áp dụng: Duyệt đồ thị theo chiều rộng từ nguồn source = 1 bằng hàng đợi CircularQueue. Khởi tạo distance = -1, parent = 0; gán distance[source] = 0 và nạp nguồn vào hàng đợi. Lần lượt lấy u ra mở rộng các đỉnh kề v: nếu distance[v] == -1 thì cập nhật distance[v] = distance[u] + 1, parent[v] = u và đưa v vào hàng đợi. Sau đó truy vết từ từng đỉnh v về nguồn theo parent rồi reverse để lấy tuyến đường xuôi. Điều kiện: Đồ thị không trọng số hoặc các cạnh đồng nhất 1 bước chuyển.

Lời gọi và kết quả kiểm thử: Chạy lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=bfs. Kết quả: ĐẠT toàn bộ kiểm tra khoảng cách và lộ trình: đỉnh 2 có h=1 [1, 2]; đỉnh 4 có h=2 [1, 2, 4]; đỉnh 6 có h=3 [1, 2, 4, 6]. Từ đó tính ra bộ chi phí τ chính xác: ID 1 là 3, ID 2 là 5, ID 3 là 7.

Chi phí thời gian/bộ nhớ: Thời gian: BFS cốt lõi là O(V + E); quá trình truy vết và sao chép đường đi cho mọi đỉnh mất thêm tối đa $O(V^2)$. Bộ nhớ phụ: O(V) cho hàng đợi và các mảng distance, parent; khi lưu toàn bộ danh sách routes cho V đỉnh có thể chiếm tối đa $O(V^2)$.

NV4 · Hoàn thiện knapsack(orders, costs, budget)

Ý tưởng/điều kiện áp dụng: Mô hình bài toán ba lô 0/1 bằng quy hoạch động. Sắp xếp đơn hàng theo ID tăng dần. Lập bảng table kích thước (n+1) × (budget+1) khởi tạo 0. Với đơn i và ngân sách b: skip = table[i, b+1]; take = cost <= b ? table[i, b - cost + 1] + benefit : -1; chọn table[i+1, b+1] = max(skip, take). Truy vết lùi từ (n+1, budget+1): nếu table[i+1, b+1] > table[i, b+1] thì thêm đơn i vào danh sách và trừ ngân sách b -= cost. Điều kiện: Ngân sách không âm, chi phí đơn cost > 0, mỗi đơn chọn tối đa 1 lần.

Lời gọi và kết quả kiểm thử: Chạy lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=dp. Kết quả: ĐẠT 100% các bài kiểm tra (ngân sách B = 8 cho kết quả tối ưu tập chọn [1, 2], tổng lợi ích là 14, chi phí là 8; bảng DP và truy vết hoàn toàn khớp nghiệm của thuật toán vét cạn).

Chi phí thời gian/bộ nhớ: Thời gian: O(n·B) để điền bảng quy hoạch động và O(n) truy vết, cộng O(n log n) sắp xếp tiền xử lý. Bộ nhớ phụ: O(n·B) để lưu trữ toàn bộ bảng table trong bộ nhớ.

4. Quyết định chọn đơn

Điền ngân sách, tập chọn, tổng thời gian/lợi ích. Giải thích một ô DP và một bước truy vết; đối chiếu với vét cạn.

• Tham số kết quả: Ngân sách B = 8. Tập đơn được chọn: {1, 2} (ID 1 và ID 2). Tổng thời gian thực hiện: $\tau = 3 + 5 = 8$. Tổng lợi ích đạt được: 6 + 8 = 14.
• Giải thích ô DP — F(2, 8) (trong Julia lưu tại table[3, 9]):
Ô F(2, 8) biểu diễn lợi ích tối đa khi xét trong tập 2 đơn đầu tiên {1, 2} với ngân sách thời gian b = 8:
- Phương án 1 (Không lấy đơn 2): Giữ nguyên phương án tối ưu từ hàng trước với ngân sách 8: F(1, 8) = 6.
- Phương án 2 (Lấy đơn 2): Chi phí đơn 2 là cost = 5 ≤ 8, nhận lợi ích của đơn 2 là 8, cộng với lợi ích tối ưu của các đơn còn lại với ngân sách dư 8 - 5 = 3: 8 + F(1, 3) = 8 + 6 = 14.
- Giá trị tối ưu là max(6, 14) = 14. Do đó ta ghi 14 vào ô F(2, 8).
• Giải thích các bước truy vết (Backtracking):
- Bắt đầu tại (i = 3, b = 8): F(3, 8) = 14 bằng F(2, 8) = 14, giá trị không tăng nên không chọn đơn 3; chuyển sang xét (i = 2, b = 8).
- Tại (i = 2, b = 8): F(2, 8) = 14 lớn hơn F(1, 8) = 6, chứng tỏ đơn 2 được chọn; ghi nhận ID 2, ngân sách còn lại b = 8 - 5 = 3; chuyển sang (i = 1, b = 3).
- Tại (i = 1, b = 3): F(1, 3) = 6 lớn hơn F(0, 3) = 0, chứng tỏ đơn 1 được chọn; ghi nhận ID 1, ngân sách b = 3 - 3 = 0 và dừng lại. Kết quả đảo ngược được tập chọn {1, 2}.
• Đối chiếu với phương pháp vét cạn (Brute-force): Xét toàn bộ $2^3 = 8$ tập con:
Ø (chi phí 0, lợi ích 0), {1} (chi phí 3, lợi ích 6), {2} (chi phí 5, lợi ích 8), {3} (chi phí 7, lợi ích 10), {1, 2} (chi phí 8, lợi ích 14 - Hợp lệ);
{1, 3} (chi phí 10 > 8), {2, 3} (chi phí 12 > 8), {1, 2, 3} (chi phí 15 > 8) — đều vi phạm ngân sách.
=> Tập hợp lệ tối ưu duy nhất là {1, 2} với lợi ích cực đại 14, trùng khớp tuyệt đối với nghiệm của quy hoạch động.

5. Giao và hoàn tác

Trạng thái	Queue (FIFO)	Stack (Lịch sử)	Đã dùng (τ)	Đã nhận (Lợi ích)
Sau lập kế hoạch	[1, 2]	[]	0	0
Sau giao một đơn (đơn 1)	[2]	[1]	3	6

Trạng thái	Queue (FIFO)	Stack (Lịch sử)	Đã dùng (τ)	Đã nhận (Lợi ích)
Sau hoàn tác	[1, 2]	[]	0	0

6. Đầu ra và nhận xét

Đường dẫn CSV kết quả: data/MINI001/ket_qua.csv **Link notebook:** TH_2026_10_09_Pluto.jl

Lỗi đã phát hiện và cách sửa: (1) Lỗi tái sử dụng đơn trong DP: Khi tính take = table[i+1, b - cost + 1], do truy cập cùng hàng i+1 nên đơn i có thể bị chọn nhiều lần (thành bài toán cái ba lô không giới hạn). Cách khắc phục: Đổi thành table[i, b - cost + 1] để chỉ lấy từ trạng thái i-1 đơn trước đó. (2) Lỗi lộ trình ngược trong BFS: Quá trình truy vết từ parent cho ra thứ tự từ đích về nguồn. Cách khắc phục: Bổ sung lời gọi reverse(route) trước khi gán vào routes[v]. (3) Lỗi thứ tự hoàn tác: Nếu đẩy đơn hoàn tác về cuối hàng đợi FIFO sẽ phá hỏng thứ tự ưu tiên ban đầu. Cách khắc phục: Đẩy đơn hoàn tác vào đầu hàng đợi FIFO (pushfirst!).

Một chức năng thay đổi ảnh hưởng đến chức năng sau như thế nào?: Các chức năng trong chuỗi ứng dụng phụ thuộc tuần tự chặt chẽ: Đầu ra của BFS quyết định chi phí τ, từ đó trực tiếp ảnh hưởng đến quyết định chọn đơn của DP. Ví dụ, khi mạng lưới bản đồ bị cắt bỏ hai cạnh (2, 4) và (3, 4), lộ trình BFS tới đỉnh 4 phải vòng qua 1 -> 3 -> 5 -> 6 -> 4 (4 cạnh), khiến chi phí τ của đơn 2 tăng từ 5 lên 9 (2×4+1=9). Do $\tau_2 = 9$ vượt quá ngân sách B = 8, đơn 2 lập tức bị DP loại bỏ, khiến kết quả tối ưu chuyển hẳn sang chọn duy nhất đơn 3 (chi phí 7, lợi ích 10) thay vì tập {1, 2} ban đầu.

Câu hỏi cuối buổi

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

• Lý do không được dùng dãy sắp giảm theo lợi ích/τ: Thuật toán tìm kiếm nhị phân hoạt động dựa trên việc so sánh khóa ở phần tử chính giữa (mid) với khóa cần tìm để loại bỏ một nửa khoảng tìm kiếm ở mỗi bước lặp. Cơ chế này chỉ hợp lệ khi toàn bộ mảng đã được sắp xếp tăng dần đơn điệu theo đúng trường khóa tìm kiếm (ở đây là trường ID). Nếu mảng được sắp theo tỷ số lợi ích/τ, các giá trị ID sẽ nằm rải rác lộn xộn không còn trật tự đơn điệu; khi so sánh khóa mục tiêu với a[mid].id, ta không thể xác định phần tử cần tìm nằm ở nửa bên trái hay nửa bên phải của mid, dẫn đến việc cắt bỏ sai nhánh và làm hàm trả về kết quả sai (bỏ sót phần tử dù nó tồn tại).
• Tiền điều kiện của hàm tìm kiếm nhị phân:
1. Cấu trúc lưu trữ dữ liệu phải cho phép truy xuất ngẫu nhiên tới bất kỳ vị trí phần tử nào với thời gian O(1) (mảng hoặc Vector).
2. Toàn bộ các phần tử trong danh mục bắt buộc phải được sắp xếp theo thứ tự tăng dần (hoặc không giảm) dựa trên trường khóa tìm kiếm (order.id).
3. Mỗi phần tử trong danh mục phải sở hữu một khóa định danh duy nhất và dữ liệu phải được cố định (không bị thêm, sửa, xóa) trong suốt phiên tra cứu.

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

• Giải thích từng thành phần của thời gian thực hiện τ đối với đơn ID 2:
- Tuyến đường ngắn nhất do BFS tìm được từ kho 1 tới điểm giao 4 là 1 -> 2 -> 4, bao gồm h = 2 cạnh.
- Thời gian di chuyển khứ hồi (cả lượt đi và lượt quay về kho): Với giả định mỗi cạnh tiêu tốn 1 đơn vị thời gian, tổng thời gian đi và về là 2 × h = 2 × 2 = 4 đơn vị.
- Thời gian tác nghiệp giao hàng tại chỗ (service): Được đặc tả cố định là 1 đơn vị thời gian.

- Tổng thời gian hoàn tất chuyển giao: $\tau = 2h + service = 4 + 1 = 5$ đơn vị thời gian.

• Khả năng tối ưu của BFS khi các cạnh mang trọng số thời gian khác nhau:

- BFS thông thường KHÔNG CÒN bảo đảm tìm được tuyến đường có thời gian ngắn nhất nữa.
- Lý do: Thuật toán BFS chỉ tối ưu hóa số lượng cạnh di chuyển (áp dụng cho đồ thị không trọng số hoặc các cạnh có trọng số bằng nhau). Khi các cạnh mang trọng số thời gian khác nhau, một đường đi có ít cạnh hơn hoàn toàn có thể tiêu tốn tổng thời gian lớn hơn nhiều so với một đường đi qua nhiều cạnh nhưng các cạnh đó có thời gian di chuyển rất ngắn. Để giải bài toán đường đi ngắn nhất theo tổng trọng số thời gian trên đồ thị có trọng số không âm, ta bắt buộc phải sử dụng thuật toán Dijkstra thay thế cho BFS.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

• Hai phương án hình thành ô F(2, 8):

1. Phương án không chọn đơn hàng 2 (Skip): Kế thừa giá trị tối ưu tốt nhất chỉ xét trong tập các đơn hàng từ 1 đến 1 với cùng mức ngân sách 8, nhận giá trị F(1, 8) = 6.
2. Phương án chấp nhận chọn đơn hàng 2 (Take): Do chi phí đơn 2 là cost = 5 ≤ 8, ta nhận được lợi ích của đơn 2 là 8, cộng với giá trị tối ưu tốt nhất từ hàng trước với phần ngân sách còn lại 8 - 5 = 3: 8 + F(1, 3) = 8 + 6 = 14.

- Giá trị tối ưu là cực đại giữa hai phương án: F(2, 8) = max(6, 14) = 14.

• Vị trí trong bảng dữ liệu Julia:

- Julia sử dụng cơ chế đánh chỉ số mảng bắt đầu từ 1 (1-based indexing). Dòng ứng với 0 đơn hàng nằm ở chỉ số 1, và cột ứng với mức ngân sách 0 nằm ở chỉ số 1.
- Do đó, ô trạng thái F(i = 2, b = 8) được ánh xạ chính xác vào tọa độ table[2 + 1, 8 + 1] = table[3, 9].

• Lý do nhánh lấy đơn phải đọc từ hàng phía trước:

- Bản chất của bài toán cái ba lô 0/1 là mỗi đơn hàng chỉ được phép giao tối đa 1 lần duy nhất. Khi đã chọn đơn i, phần ngân sách còn lại (b - cost) chỉ được phép dùng để chọn các đơn hàng chưa từng xét từ 1 đến i-1 (nằm ở hàng i trong bảng Julia).
- Nếu đọc dữ liệu từ cùng hàng hiện tại (table[i+1, b - cost + 1]), đơn hàng i hoàn toàn có thể được chọn lặp lại thêm nhiều lần nữa, biến bài toán thành dạng cái ba lô không giới hạn (Unbounded Knapsack) và sai lệch hoàn toàn với yêu cầu thực tế.

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

• Sự biến đổi trạng thái sau khi giao ID 1 rồi hoàn tác ngay:

- Sau khi giao đơn 1: Đơn 1 được rút khỏi đầu hàng đợi FIFO -> FIFO còn [2]; Đơn 1 được đẩy vào đỉnh ngăn xếp Stack -> Stack trở thành [1]; Tổng thời gian đã dùng tăng thêm 3 -> Thời gian = 3; Tổng lợi ích thu về tăng thêm 6 -> Lợi ích = 6.
- Sau khi hoàn tác ngay: Đơn 1 được rút khỏi đỉnh ngăn xếp Stack -> Stack trở về rỗng []; Đơn 1 được trả ngược lại vào vị trí đầu của hàng đợi FIFO -> FIFO khôi phục lại trọn vẹn [1, 2]; Thời gian đã dùng giảm đi 3 -> Thời gian = 0; Lợi ích thu về giảm đi 6 -> Lợi ích = 0. Toàn bộ trạng thái trở về nguyên vẹn như lúc vừa lập kế hoạch xong. (Lưu ý: Phải đưa đơn hoàn tác về đầu FIFO mới bảo toàn đúng thứ tự ưu tiên giao hàng, nếu đưa về cuối FIFO sẽ làm xáo trộn trật tự ban đầu).

• Bất biến trạng thái (Invariants) kiểm tra không mất hoặc lặp đơn:

1. Tính rời rạc: Tập các mã đơn trong hàng đợi FIFO và tập các mã đơn trong ngăn xếp Stack luôn có giao bằng rỗng (FIFO ∩ Stack = ∅).
2. Bảo toàn số lượng: Hợp của hai tập trên luôn bằng chính xác tập hợp các đơn hàng tối ưu đã được thuật toán DP phê duyệt ban đầu (FIFO ∪ Stack = Plan).
3. Tính nhất quán đại lượng: Tổng thời gian đã dùng và tổng lợi ích đã nhận tại mọi thời điểm luôn bằng chính xác tổng chi phí và tổng lợi ích của các đơn hàng đang hiện diện trong ngăn xếp Stack.

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

• Bảng tổng hợp độ phức tạp theo n, V, E, B:

- Sắp xếp trộn (Merge Sort): Thời gian O(n log n); Bộ nhớ phụ O(n).
- Tìm kiếm nhị phân (Binary Search): Thời gian O(log n); Bộ nhớ phụ O(1).
- BFS cốt lõi (Core BFS): Thời gian O(V + E); Bộ nhớ phụ O(V) cho hàng đợi và mảng parent. Khi tính cả bước truy vết và lưu trữ tường minh danh sách lộ trình routes cho toàn bộ V đỉnh thì tổng độ dài các đường đi có thể khiến thời gian và bộ nhớ phụ tăng lên mức $O(V^2)$ trong trường hợp đồ thị suy biến dạng đường thẳng dài. (Cần phân biệt rõ bộ nhớ lưu tuyến này với BFS chỉ lưu parent).
- Quy hoạch động bảng đầy đủ (Full-table Knapsack DP): Thời gian O(n·B); Bộ nhớ phụ O(n·B) để duy trì toàn bộ bảng phương án kích thước (n+1) × (B+1).

• Lý do không thể khẳng định mọi cây tìm kiếm nhị phân (BST) đều tra cứu trong O(log n):

- Chi phí thời gian của phép tra cứu trên một cây BST phụ thuộc trực tiếp vào chiều cao h của cây, tức là O(h), chứ không mặc định phụ thuộc vào số lượng nút n.
- Độ phức tạp O(log n) chỉ đạt được khi cây có cấu trúc cân bằng lý tưởng (chiều cao $h \approx \log_2 n$).
- Đối với một cây BST thông thường không có cơ chế tự cân bằng, nếu dữ liệu các khóa được chèn theo một thứ tự đã sắp sẵn (tăng dần hoặc giảm dần), cây sẽ bị thoái hóa hoàn toàn thành một đường thẳng (danh sách liên kết đơn). Khi đó, chiều cao của cây chạm mức xấu nhất là h = O(n), và thao tác tra cứu trên cây sẽ suy biến thành tìm kiếm tuần tự với độ phức tạp O(n) thay vì O(log n).